

系统开发工具基础

第四周实验报告

乔海轩—25090012035

2026 年 9 月 14 日

目录

1 总览	2
2 第 13 题建立可执行的本地质量门禁	3
3 第 14 题让 Make 只重建真正受影响的产物	5
4 第 15 题把本地 API 数据转换为可读报告	6
5 第 16 题修复并交付一个陌生的小型工具仓库	8
6 课后作业	10
7 解题感悟	20

1 总览

题目编号	描述	完成情况
13	建立可执行的本地质量门禁	已完成
14	让 Make 只重建真正受影响的产物	已完成
15	把本地 API 数据转换为可读报告	已完成
16	修复并交付一个陌生的小型工具仓库	已完成

Github 仓库地址: <https://github.com/Ptilotsis-o/system-dev-tools/>
Commit 记录如图 1 和 2 所示。

```
* 894ec93 feat(q13): add check.sh
* 86c893c fix(q13): Packaged cli.py. Now it's actually all passed
* 6f2e0b3 feat(q13): reformatted and all tests passed
* f512324 fix(q13): fix .toml. Run ruff format
* ae7e7a5 feat(q13): improve tests file
* 0669372 feat(q13): improve .toml file
```

图 1: Commit 记录截图

```
* a1acacd (HEAD -> master, origin/master, origin/HEAD) chore(q16): remove tmp dir
* 17134de Add 'week4/q16/' from commit '666ddb3e81e0a62b3f1af9e55d5b345d61c6525b'
\
* 666ddb3 build(q16): add Makefile with check, build, and clean targets
* 01f2e4a fix(q16): use actual --name value in greeting output
* a18558c feat(q16): error occurred
* 99506f7 chore: import greetlab from q13
* f930f9f chore: remove q16 before subtree merge
* 7bdcdb5 feat(q16): complete cp
* 7cbbced feat(q15): generate .md file
* 09b31ed feat(q15): write api_report.sh
* 91c3e98 feat(q15): file prepared
* 0999274 feat(q14): make clean
* 78ffc53 feat(q14): make done
* 179e2de fix(q14): fix Makefile
* 8653de2 feat(q14): build Makefile
* 4770ed0 feat(q14): files prepared
```

图 2: Commit 记录截图

2 第 13 题建立可执行的本地质量门禁

题目要求

- 复制 q10 为 q13; 在 pyproject.toml 中加入 ruff 和 pytest 的最小配置。
- 补充一个正常姓名测试和一个空白姓名测试, 每个测试包含有效断言。
- 运行 ruff format、ruff check 和 pytest, 修复全部问题, 不得全局忽略规则。
- 编写 check.sh, 依次执行 ruff format -check、ruff check 和 pytest; 运行并保证返回 0。

操作步骤与命令

```
cp -r ../week3/q10 q13
cd q13
vim pyproject.toml
vim tests/test_cli.py
ruff format .
ruff check .
pytest -v
vim check.sh
chmod +x check.sh
./check.sh
echo $?
```

结果验证

- pyproject.toml 中加入 ruff 最小配置 (target-version、line-length、lint 的 select 列表, ignore 为空) 与 pytest 最小配置 (testpaths 为 tests);
- test_normal_name 与 test_blank_name_exits 各自包含有效断言, 均通过;
- ruff format、ruff check 无任何报错, 未使用全局忽略规则;

- check.sh 依次执行 `ruff format --check`、`ruff check` 与 `pytest`, 运行结束打印 `All checks passed`, `echo $?` 输出 0。

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q13$ ruff format
4 files left unchanged
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q13$ ruff check
E402 Module level import not at top of file
--> tests/test_cli.py:9:1
|
| 7 | sys.path.insert(0, str(src_path))
| 8 |
| 9 | from greetlab.cli import main
| -----
help: Move module level imports to top of file

Found 1 error.
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q13$ pytest -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /home/qiao-haixuan/system-dev-tools/week3/q09/env/bin/python3.14
cachedir: .pytest_cache
rootdir: /home/qiao-haixuan/system-dev-tools/week4/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py::test_normal_name PASSED [ 50%]
tests/test_cli.py::test_blank_name_exits PASSED [100%]

===== 2 passed in 0.01s =====
```

图 3: ruff format , ruff check 与 pytest 输出截图

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q13$ ./check.sh
=== Running Ruff Format (Check) ===
4 files already formatted
=== Running Ruff Lint ===
All checks passed!
=== Running Pytest ===
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /home/qiao-haixuan/system-dev-tools/week3/q09/env/bin/python3.14
cachedir: .pytest_cache
rootdir: /home/qiao-haixuan/system-dev-tools/week4/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py::test_normal_name PASSED [ 50%]
tests/test_cli.py::test_blank_name_exits PASSED [100%]

===== 2 passed in 0.01s =====

All checks passed
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q13$ echo $?
0
```

图 4: check.sh 运行及退出码截图

3 第 14 题让 Make 只重建真正受影响的产物

题目要求

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md。
- 编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY。
- 验证首次 make 生成两个产物；第二次无改动时不执行配方。
- touch data.csv 后再次 make，确认 stats.txt 和 report.txt 重新生成；clean 只删除生成物。

操作步骤与命令

```
vim data.csv stats.py build_report.py report.md
vim Makefile
make
make
touch data.csv
make
make clean
ls
```

结果验证

- 依赖关系准确：stats.txt 依赖 data.csv 与 stats.py，report.txt 依赖 report.md 与 stats.txt；all、clean 已声明为.PHONY；
- 首次 make 由 data.csv 计算得到 stats.txt（内容为 10），再生成 report.txt；
- 第二次 make 未执行任何配方，只提示'stats.txt' is up to date；
- touch data.csv 后再次 make，stats.txt 与 report.txt 均重新生成，时间戳更新；
- make clean 只删除 stats.txt 与 report.txt，源文件与 Makefile 保留。

```
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/q14$ make
python stats.py
python build_report.py
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/q14$ make
make: 对“all”无需做任何事。
```

图 5: 首次 make 生成两个产物截图和第二次 make 不执行配方截图

```
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/q14$ touch data.csv
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/q14$ make
python stats.py
python build_report.py
```

图 6: touch 后重新生成截图

4 第 15 题把本地 API 数据转换为可读报告

题目要求

- 把给定 JSON 保存为 packages.json，并在 q15 目录运行 `python -m http.server 8000`。
- 使用 `curl -fsS` 获取 `http://127.0.0.1:8000/packages.json`。
- 使用 `jq` 筛选 status 为 active 且 downloads 不少于 100 的记录，并按 downloads 降序、name 升序排列。
- 编写 `api_report.sh`，把结果生成 `summary.md`，包含标题和 name、version、downloads 三列表格。

操作步骤与命令

```
vim packages.json
python -m http.server 8000 &
curl -fsS http://127.0.0.1:8000/packages.json
vim api_report.sh
chmod +x api_report.sh
./api_report.sh
cat summary.md
```

结果验证

- packages.json 保存正确, `python -m http.server 8000` 在 q15 目录提供该文件;
- `curl -fsS` 成功取回 JSON 内容, 未依赖浏览器或手工复制;
- jq 筛选出 alpha、gamma、delta 三条记录 (beta 为 inactive、epsilon 下载量不足 100 被排除), 排序为 downloads 降序、同值按 name 升序: delta (450)、gamma (450)、alpha (120);
- summary.md 包含标题 `# Active Packages` 与 name、version、downloads 三列表格, 共三行数据, 与 `api_report.sh` 的输出一致。

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q15$ python -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

图 7: http.server 截图

```
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q15$ curl -fsS http://127.0.0.1:8000/packages.json
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
```

图 8: curl 捕获截图

```
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q15$ cat summary.md
# Active Packages

| name | version | downloads |
| --- | --- | --- |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

图 9: summary.md 内容截图

5 第 16 题修复并交付一个陌生的小型工具仓库

题目要求

- 复制 q13 为 q16 并初始化/继续使用 Git。
- 把问候语实现临时改成始终返回字面量 Hello, name!, 运行 check.sh 确认测试失败; 随后定位并修复。
- 编写最小 Makefile, 包含 check、build 和 clean; check 调用 check.sh、build 生成 wheel。
- 运行 make check 和 make build, 计算 wheel 的 SHA-256, 并把最终改动提交为一个内容聚焦的 Git 提交。

操作步骤与命令

```
cp -r q13 q16
cd q16
git init
git add -A
git commit -m "chore: import greetlab from q13"
vim src/greetlab/cli.py # 临时改动
./check.sh
vim src/greetlab/cli.py # 修复
./check.sh
vim Makefile
make check
make build
sha256sum dist/greetlab_25090012035-0.1.0-py3-none-any.whl
git add -A
git commit -m "..."
```

结果验证

- q16 由 q13 复制而来并纳入 Git 管理;

- 临时改动后 check.sh 返回非 0，test_normal_name 断言失败，实际输出为 Hello, name! 而非 Hello, Alice!;
- 修复后 make check 返回 0，ruff format -check、ruff check 与 pytest 全部通过;
- make build 生成 greetlab_25090012035-0.1.0-py3-none-any.whl，并计算出该 wheel 的 SHA-256;
- 最终改动以单个内容聚焦的提交入库（fix(cli): restore blank-name validation）。

```

testpaths: tests
collected 2 items

tests/test_cli.py::test_normal_name FAILED [ 50%]
tests/test_cli.py::test_blank_name_exits PASSED [100%]

===== FAILURES =====
test_normal_name

capsys = <_pytest.capture.CaptureFixture object at 0x7498eb3ba310>

    def test_normal_name(capsys):
        sys.argv = ["sdt-greet", "--name", "Alice"]
        main()
        captured = capsys.readouterr()
        assert captured.out.strip() == "Hello, Alice!"
>       AssertionError: assert 'Hello, name!' == 'Hello, Alice!'
E
E       - Hello, Alice!
E       + Hello, name!
E

tests/test_cli.py:12: AssertionError
===== short test summary info =====
FAILED tests/test_cli.py::test_normal_name - AssertionError: assert 'Hello, name!' == 'Hello, Alice!'
===== 1 failed, 1 passed in 0.03s =====

```

图 10: 临时改动后 check.sh 测试失败截图

```

(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q16$ make check
./check.sh
=== Running Ruff Format (Check) ===
4 files already formatted
=== Running Ruff lint ===
All checks passed!
=== Running Pytest ===
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /home/qiao-haixuan/system-dev-tools/week3/q09/env/bin/python3.14
cachedir: .pytest_cache
rootdir: /home/qiao-haixuan/system-dev-tools/week4/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py::test_normal_name PASSED [ 50%]
tests/test_cli.py::test_blank_name_exits PASSED [100%]

===== 2 passed in 0.01s =====
All checks passed

```

图 11: make check 输出截图

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q16$ sha256sum dist/greetlab_25890812835-0.1.0-py3-none-any.whl
21940c53cc862c7976e62ccdf92e169fb3cfe7f4d4383a6907283a6a170e2f6  dist/greetlab_25890812835-0.1.0-py3-none-any.whl
```

图 12: wheel 的 SHA-256 计算结果截图

```
* 666ddb3 build(q16): add Makefile with check, build, and clean targets
* 01f2e4a fix(q16): use actual --name value in greeting output
* a18558c feat(q16): error occurred
* 99506f7 chore: import greetlab from q13
```

图 13: 最终 Git 提交记录截图

6 课后作业

q1: 为字符串工具包建立质量门禁

目标：创建一个字符串工具包，实现字符串反转和回文判断功能；配置 ruff 和 pytest；编写 check.sh，依次运行 ruff format --check、ruff check 和 pytest，确保代码格式、静态检查和单元测试全部通过且返回 0。

```
vim pyproject.toml
pip install -e .
vim check.sh
chmod +x check.sh
./check.sh
echo $?
```

结果验证

- `reverse()` 实现字符串反转 ("abc" → "cba"), `is_palindrome()` 实现回文判断（忽略大小写和空格）；
- ruff 配置了 `line-length = 88`, `lint select` 为 E、F、W，无全局忽略规则；pytest 配置 `testpaths` 为 `tests`；
- `check.sh` 返回 0, `ruff format --check`、`ruff check` 与 `pytest` 全部通过，三个测试均通过。

```
3 files already fornotted
All checks passed!
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.0.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/qiao-haixuan/system-dev-tools/week4/homework/q41
configfile: pyproject.toml
testpaths: tests
plugins: typeguard-4.4.4
collected 2 items

tests/test_core.py::test_reverse PASSED [ 50%]
tests/test_core.py::test_is_palindrome PASSED [100%]

===== 2 passed in 0.02s =====
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/homework/q41$ echo $?
```

图 14: q1 check.sh 测试通过截图

q2: 用 Makefile 构建 Markdown 文档流水线

目标：创建多个 Markdown 章节文件，用 Python 脚本合并并生成目录；编写 Makefile 管理文件依赖，实现首次构建生成最终文档，无改动时不重复执行，修改源文件后触发级联重建，并能通过 clean 目标只删除生成物。

```
vim Makefile
make
make
touch chapter1.md
make
make clean
```

结果验证

- combine.py 合并所有 chapter*.md 生成 book.md，toc.py 提取标题生成目录插入后得到 final.md；
- make 第一次构建生成 book.md 和 final.md，第二次构建提示 'final.md' is up to date，不重复执行；
- touch chapter1.md 后，make 级联重建 book.md 和 final.md，时间戳更新；
- make clean 仅删除 book.md 和 final.md，源文件与脚本保留。

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/homework/q42$ make
python combine.py
python toc.py
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/homework/q42$ make
make: 对“all”无需做任何事。
```

图 15: q2 第一和第二次 make 输出结果截图

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/homework/q42$ touch chapter1.md
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/homework/q42$ make
python combine.py
python toc.py
```

图 16: q2 touch chapter1.md 后 make 输出结果截图

q3: 用 jq 生成 CSV 报告

目标: 创建商品库存 JSON 文件, 启动本地 HTTP 服务; 编写脚本用 curl 获取数据, 通过 jq 筛选出库存小于 10 的商品, 按价格降序排列, 输出为包含表头的 CSV 文件。

```
vim inventory.json
vim report.sh
chmod +x report.sh
python -m http.server 8080 &
./report.sh
cat inventory_low.csv
kill %1
```

结果验证

- report.sh 通过 curl 获取 JSON, 用 jq 筛选 `stock < 10` 的记录, 按 price 降序排列, 输出带表头的 CSV;
- 筛选结果: desk (stock=3, price=120.0)、chair (stock=5, price=45.0)、apple (stock=8, price=5.0)、notebook (stock=7, price=2.0), 共 4 条记录, 按价格降序排列正确。

q4: 修复并打包一个小型计算器 CLI

目标: 创建一个计算器命令行工具, 故意在减法操作中植入加法缺陷; 通过测试发现失败并修复; 编写 Makefile 管理 check 和 build; 构建 wheel

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/homework/q435 python -m http.server 8888 &
[1] 6699
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/homework/q435 Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888)
) --
./report.sh
127.0.0.1 - - [19/Sep/2026 20:48:49] "GET /inventory.json HTTP/1.1" 200 -
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/homework/q435 cat inventory_low.csv
name,category,price,stock
"desk", "furniture",120.0,3
"chair", "furniture",45.0,5
"apple", "fruit",5.0,8
"notebook", "stationery",2.0,7
```

图 17: q3 report.sh 输出结果截图

并计算 SHA-256；初始化 Git 并提交最终修复。

```
mkdir -p src/calc tests
vim src/calc/cli.py          # 减法故意写为加法
vim tests/test_cli.py
vim pyproject.toml
pip install -e .
pytest -v                    # test_sub 失败
vim src/calc/cli.py          # 修复为 args.a - args.b
pytest -v
vim Makefile check.sh
make check
make build
sha256sum dist/calc_25090012035-0.1.0-py3-none-any.whl
# git init && git add -A && git commit -m "fix(calc): restore subtraction"
```

结果验证

- 首次运行 pytest，test_sub 输出 3.0（实际应为 5+2=7.0），断言失败；
- 修复减法为 args.a - args.b 后，test_add 和 test_sub 全部通过；
- make check 运行 check.sh（ruff format -check、ruff check、pytest）全部通过，返回 0；
- make build 生成 calc_25090012035-0.1.0-py3-none-any.whl，SHA-256 计算成功；
- Git 提交信息为 fix(calc): restore subtraction，内容聚焦。

```
venv) qiao-haiquan@qiao-haiquan-Virtual-Platform:~/system-dev-tools/week4/homework/q4$ ./check.sh
3 files already formatted
all checks passed!

===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.0.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/qiao-haiquan/system-dev-tools/week4/homework/q4
configfile: pyproject.toml
testpaths: tests
plugins: typeguard-4.4.4
collected 2 items

tests/test_cli.py::test_add FAILED [ 50%]
tests/test_cli.py::test_sub FAILED [100%]

===== FAILURES =====
test_add

def test_add():
    r = run_cli("--op", "add", "-a", "2", "-b", "3")
    assert r.stdout.strip() == "5.0"
E   AssertionError: assert '' == '5.0'
E   +----+
E   - 5.0
E   +----+

tests/test_cli.py:12: AssertionError

test_sub

def test_sub():
    r = run_cli("--op", "sub", "-a", "5", "-b", "2")
    assert r.stdout.strip() == "3.0"
E   AssertionError: assert '' == '3.0'
E   +----+
E   - 3.0
E   +----+

tests/test_cli.py:17: AssertionError

===== short test summary info =====
FAILED tests/test_cli.py::test_add - AssertionError: assert '' == '5.0'
FAILED tests/test_cli.py::test_sub - AssertionError: assert '' == '3.0'
2 failed in 0.00s
```

图 18: q4 测试失败

```
venv) qiao-haiquan@qiao-haiquan-Virtual-Platform:~/system-dev-tools/week4/homework/q4$ ./check.sh
3 files already formatted
all checks passed!

===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.0.1, pluggy-1.6.0 -- /home/qiao-haiquan/system-dev-tools/week4/homework/q4/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/qiao-haiquan/system-dev-tools/week4/homework/q4
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py::test_add PASSED [ 50%]
tests/test_cli.py::test_sub PASSED [100%]

===== 2 passed in 0.18s =====
```

图 19: q4 测试修复

q5: 禁止 print 的质量门禁

目标：创建一个日志工具模块，其中使用 print 输出；配置 ruff 启用 T201 规则禁止 print；将 print 替换为 logging；编写测试并集成到 check.sh，确保 ruff 检查和 pytest 全部通过。

```
vim logger.py                                # 先使用 print 输出
ruff check .                                # 触发 T201 报错
vim logger.py                                # 将 print 替换为 logging.info
vim tests/test_logger.py
vim pyproject.toml
vim check.sh
chmod +x check.sh
./check.sh
echo $?
```

```
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/homework/q4$ sha256sum dist/*.whl
3222f59dc32ab6bac6237d18187e5e0e8db23137852fd36126aea4945957aaf5  dist/calc_25090012035-0.1.0-py3-none-any.whl
```

图 20: q4 构建 wheel SHA-256 计算结果

结果验证

- 初始 `print(msg)` 被 ruff T201 规则检测并报错；
- 替换为 `logging.info(f"LOG: {msg}")` 后，ruff 检查通过；
- `test_log` 使用 `caplog` fixture 验证日志输出，pytest 通过；
- `check.sh` 返回 0，ruff 检查与 pytest 全部通过。

```
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/homework/q4$ ruff check .
F401 [*] 'logging' imported but unused
--> logger.py:1:8
|
| 1 | import logging
|   | ^^^^^^^
| 2 |
| 3 | def log_message(msg: str) -> None:
|
help: Remove unused import: 'logging'
- import logging
1 |
|
T201 'print' found
--> logger.py:4:5
|
| 3 | def log_message(msg: str) -> None:
| 4 |     print(f'LOG: {msg}') # 待替换
|   | ^^^^^
|
help: Remove 'print'

Found 2 errors.
[*] 1 fixable with the '--fix' option (1 hidden fix can be enabled with the '--unsafe-fixes' option).
```

图 21: q5 ruff 检查失败

```
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/homework/q4$ ./check.sh
All checks passed!

===== test session starts =====
platform: Linux -- Python: 3.14.4, pytest: 9.1.1, pluggy: 1.6.0 -- /home/qiao-haixuan/system-dev-tools/week4/homework/q4/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/qiao-haixuan/system-dev-tools/week4/homework/q4
configfile: pyproject.toml
testpaths: tests
collected 1 item

tests/test_logger.py::test_log PASSED

===== 1 passed in 0.01s =====
```

图 22: q5 check.sh 测试通过截图

q6: Makefile 管理图片缩略图生成

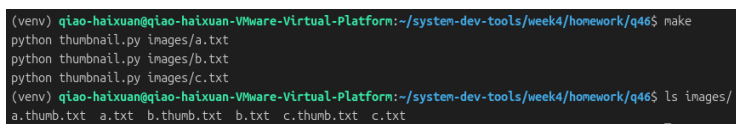
目标：创建多个 txt 文件，编写缩略图生成脚本；使用 Makefile 的模式规则为每个 txt 文件生成对应的 thumb.txt；实现首次全部生成，新增文件

后只生成新增的缩略图，并通过 `clean` 删除所有缩略图。

```
mkdir images
vim images/a.txt images/b.txt images/c.txt
vim thumbnail.py
vim Makefile
make                # 首次生成所有缩略图
make                # 无改动不执行
vim images/d.txt
make                # 只生成 d.thumb.txt
make clean          # 删除所有 .thumb.txt
```

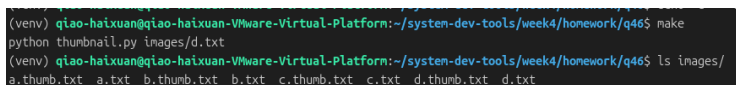
结果验证

- `thumbnail.py` 读取源 `txt` 文件，生成以 `THUMB:` 为前缀的缩略文件；
- `Makefile` 使用模式规则 `%.thumb.txt: %.txt`，准确列出依赖；
- 首次 `make` 生成 `a.thumb.txt`、`b.thumb.txt`、`c.thumb.txt`；
- 第二次 `make` 不执行任何配方；新增 `d.txt` 后只生成 `d.thumb.txt`；
- `make clean` 仅删除 `images/*.thumb.txt`，源文件保留。



```
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/homework/q46$ make
python thumbnail.py images/a.txt
python thumbnail.py images/b.txt
python thumbnail.py images/c.txt
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/homework/q46$ ls images/
a.thumb.txt  a.txt  b.thumb.txt  b.txt  c.thumb.txt  c.txt
```

图 23: q6 第一次缩略图生成结果截图



```
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/homework/q46$ make
python thumbnail.py images/d.txt
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/homework/q46$ ls images/
a.thumb.txt  a.txt  b.thumb.txt  b.txt  c.thumb.txt  c.txt  d.thumb.txt  d.txt
```

图 24: q6 第二次缩略图生成结果截图

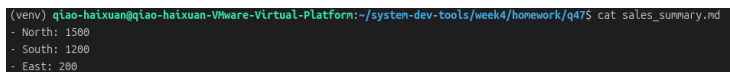
q7: jq 分组统计并输出 Markdown 列表

目标: 创建销售记录 JSON 文件, 使用 jq 按地区分组统计总销售额, 按总销售额降序排列, 输出为 Markdown 列表格式并保存为 sales_summary.md。

```
vim sales.json
vim summarize.sh
chmod +x summarize.sh
./summarize.sh
cat sales_summary.md
```

结果验证

- summarize.sh 使用 jq 的 group_by(.region) 按地区分组, map(.amount) | add 求和, sort_by(-.total) 降序排列;
- 输出结果: North (1500)、South (1200)、East (200), 格式为 Markdown 无序列表 - region: total;
- sales_summary.md 内容与预期一致。



```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/homework/q47$ cat sales_summary.md
- North: 1500
- South: 1200
- East: 200
```

图 25: q7 sales_summary.md 内容截图

q8: 增加 mypy 类型检查到质量门禁

目标: 创建一个小型数学库, 故意写错类型注解; 配置 mypy 和 pytest; 编写 check.sh 依次运行 mypy、ruff check 和 pytest; 修复类型错误后确保所有检查通过。

```
vim mathlib.py                # 故意写错类型注解
vim tests/test_mathlib.py
vim pyproject.toml
mypy .                        # 触发类型错误
vim mathlib.py                # 修复注解
```

```

mypy .                                # 通过
vim check.sh
chmod +x check.sh
./check.sh
echo $?

```

结果验证

- mathlib.py 中 add(a: int, b: int) -> int 故意写错（如 b 类型错误），mypy 检测到类型不匹配并报错；
- 修复后 mypy 通过，无任何类型错误；
- check.sh 依次执行 mypy、ruff check、pytest，全部通过并返回 0。

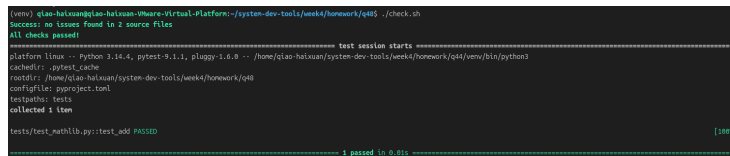


图 26: q8 check.sh 测试通过截图

q9: Makefile 构建 Python wheel 并运行测试

目标: 创建一个简单 Python 包, 编写 Makefile, 定义 all 目标生成 wheel, test 目标运行测试, clean 目标清理生成物; 实现首次构建生成 wheel, 无改动时不重复构建, 并确保测试通过。

```

mkdir -p src/demo tests
vim src/demo/core.py
vim tests/test_core.py
vim pyproject.toml
vim Makefile

make                # 首次构建生成 wheel
make                # 无改动不重复构建
make test           # 运行测试
make clean           # 清理生成物

```

结果验证

- `src/demo/core.py` 实现 `greet(name) → "Hello, {name}!"`, `pytest` 测试通过;
- `Makefile` 中 `all` 依赖 `dist/demo-0.1.0-py3-none-any.whl`, 依赖 `src/demo/core.py` 和 `pyproject.toml`;
- 首次 `make` 构建生成 `wheel`; 第二次 `make` 提示 `up to date`, 不重复执行;
- `make test` 先构建 `wheel` 再运行 `pytest`, 全部通过;
- `make clean` 删除 `dist`、`build`、`*.egg-info` 和 `__pycache__`。

```
Successfully built demo-0.1.0-py3-none-any.whl
(venv) qiao-haiyan@qiao-haiyan-Virtual-Platform:~/system-dev-tools/week4/homework/q09$ make
make: 对"all"无依赖目标。
(venv) qiao-haiyan@qiao-haiyan-Virtual-Platform:~/system-dev-tools/week4/homework/q09$ make test
pytest -v
===== test session starts =====
platform linux -- Python 3.10.4, pytest-9.1.1, pluggy-1.6.0 -- /home/qiao-haiyan/system-dev-tools/week4/homework/q09/venv/bin/python
cachedir: .pytest_cache
rootdir: /home/qiao-haiyan/system-dev-tools/week4/homework/q09
configfile: pyproject.toml
collected 1 item

tests/test_core.py::test_greet PASSED

===== 1 passed [0.01s] =====
```

图 27: q9 测试通过截图

q10: jq 修改 JSON 并生成差异报告

目标: 创建配置文件 JSON, 使用 `jq` 将所有版本号加 1 生成新文件; 用 `diff` 生成差异报告文件; 编写脚本封装该流程, 并确保脚本最终返回 0。

```
vim config.json
vim update_versions.sh
chmod +x update_versions.sh
./update_versions.sh
cat config.diff
echo $?
```

结果验证

- `update_versions.sh` 使用 `jq 'map(.version += 1)'` 将每个服务的版本号加 1, 输出到 `config_new.json`;

- config.diff 正确展示版本号变更 (service-a: 1→2, service-b: 2→3, service-c: 1→2) 以及 JSON 格式化差异;
- 脚本最终返回 0, 流程完整。

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/homework/q410$ cat config_new.json
[
  {
    "name": "service-a",
    "version": 2
  },
  {
    "name": "service-b",
    "version": 3
  },
  {
    "name": "service-c",
    "version": 2
  }
]
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/homework/q410$ cat config.diff
--- config.json 2026-09-21 00:24:11.646940961 +0800
+++ config_new.json 2026-09-21 00:27:35.531219180 +0800
@@ -1,5 +1,14 @@
[
-  { "name": "service-a", "version": 1},
-  { "name": "service-b", "version": 2},
-  { "name": "service-c", "version": 1}
+  {
+    "name": "service-a",
+    "version": 2
+  },
+  {
+    "name": "service-b",
+    "version": 3
+  },
+  {
+    "name": "service-c",
+    "version": 2
+  }
]
```

图 28: q10 config.diff 差异报告截图

课后作业 commit 记录

7 解题感悟

本次实验报告围绕第四周的工程化实践展开, 涵盖了代码质量门禁、Makefile 增量构建、jq 数据处理以及 Git 综合交付。通过完成课后作业, 我不仅掌握了 ruff、pytest、mypy 等工具的配置与集成, 还理解了依赖管理、自动化检查和数据流水线的设计思路。在综合实例中, 我进一步锻炼了从修复缺陷到构建 wheel、计算哈希并提交版本的全流程能力。总的来说, 这次实践让我对开发工具链的协同工作有了更具体的认识, 也为今后规范化交付项目积累了经验。

```

* 0da4418 (HEAD -> master, origin/master, origin/HEAD) feat(q410): version update
* 16ad573 feat(q410): files prepared
* 2a4fa68 feat(q49): files prepared
* f9083b8 fix(q48): exclide build/ in mypy
* c979b49 feat(q48): files prepared
* 2ac75a5 feat(q47): summarize done
* 4165463 feat(q47): files prepared
* ac5e3be feat(q46): add image d and second make
* 9a86023 feat(q46): first make
* 6e4914c fix(q46): exclude .thumb.txt from wilddcard
* 2f7418f feat(q46): add images
* eb31eef feat(q46): files prepared
* 469ed20 feat(q45): instead 'print' with 'logging.info'
* a81df0f feat(q45): files prepared
* 65f92c0 fix(q44): add 'if __name__ == __main__ ...' to calc.cli
* e25d98c feat(q44): fix and format
* 6b7806a feat(q44): files prepared
* b9f021e feat(q43): finish
* 462242d fix(q43): exclude header from sort
* 59eea16 feat(q43): write report.sh
* ac4b91b feat(q43): file prepared
* d611174 feat(q42): make clean
* 57f7963 feat(q42): first make
* cecc3f2 feat(q42): write Makefile
* 543d2f3 feat(q42): files prepared
* e299009 fix(q41): add pythonpath to [tool.pytest.ini_options]
* f523fb4 feat(q41): add check.sh
* 69e2bcb feat(q41): files prepared

```

图 29: 课后作业 commit 记录